

Module 1

Introduction to

1.1 Introduction to computers

Computers are electronic machines that run on electricity. A computer has many parts. All the parts of the computer together make the computer system. A computer system mainly comprises a central processing unit, memory, input/output devices, and storage devices. All these components function together as a single unit to deliver the desired output.

1.2 Input and output devices

❖ What are Input Devices?

The devices which are used to input the data and the program in the computer are known as "Input Devices". For example, to input text, we use a keyboard, therefore the keyboard is an input device, in the same way to input any audio or sound we use microphones, therefore, the microphone is an input device.

❖ What are Output Devices?

The devices that help in getting and viewing the desired results are called output devices.

❖ Difference Between Output Devices and Input Devices

Input Devices	Output Devices
Input is any data that is sent to the computer for processing.	Output is the result of processed data that we can see through some other devices.
Input devices are controlled by the users.	Output devices are controlled by computers.
Mouse, Keyboards etc., are input device examples.	Monitors, Printers etc., are examples of output devices.

❖ List of Input and Output Devices

There are different types of input and output devices.

Input devises	Output Devices
Keyboard	Monitor
Mouse	Printer
Joystick	Speaker
Microphone	Projector
Scanner	Headphones
Light pen	Optical mark readers

1.3 Designing efficient programs.

The program development process includes three important stages, namely, program design, program coding and program testing. All the three stages contribute to the production of high-quality programs.

❖ Program Design

Program design is the foundation for a good program. It is basically concerned with the development of a strategy to be used in writing the program, in order to achieve the solution of a problem. This includes mapping out a solution procedure and the form the program would take. The program design involves the following four stages:

1. Problem analysis.
2. Outlining the program structure.
3. Algorithm development.
4. Selection of control structures.

1.Problem Analysis

Before we think of a solution procedure to the problem, we must fully understand the nature of the problem and what we want the program to do. Without the comprehension and definition of the problem at hand, program design might turn into a hit or miss approach. We must carefully decide the following at this stage. What kind of data will go in?; What kind of outputs are needed?; and What are the constraints and conditions under which the program has to operate?

2.Outlining the Program Structure

Once we have decided what we want and what we have, then the next step is to decide how to do it. C as a structured language lends itself to a top down approach. Top down means decomposing of the solution procedure into tasks that form a hierarchical structure,

The essence of the top down design is to cut the whole problem into a number of independent constituent tasks, and then to cut the tasks into smaller subtasks, and so on, until they are small enough to be grasped mentally and to be coded easily. These tasks and subtasks can form the basis of functions in the program.

3.Algorithm Development

After we have decided a solution procedure and an overall outline of the program, the next step is to work a out a detailed definite, step by step procedure, known as algorithm for each function. The most common method of describing an algorithm is through the use of flowcharts.

4.Control Structures

A complex solution procedure may involve a large number of control statements to direct the flow of execution. In such situations, indiscriminate use of control statements such as **goto** may lead to unreadable and comprehensible programs.

It has been demonstrated that any algorithm can be structured, using the three basic control structure, namely, sequence structure, selection structure, and looping structure.

- Sequence structure denotes the execution of statements sequentially one after another.

- Selection structure involves a decision, based on a condition and may have two or more branches, which usually join again at a later point.
 - if ...else and switch statements in C can be used to implement a selection structure.
- Looping structure is used when a set of instructions is evaluated repeatedly. This structure can be implemented using do, while, or for statements.

A well designed program would provide the following benefits:

1. Coding is easy and error free.
2. Testing is simple.
3. Maintenance is easy.
4. Good documentation is possible.
5. Cost estimates can be made more accurately.
6. Progress of coding may be controlled more precisely

1.4 Introduction to C

C is a general-purpose programming language developed by **Dennis Ritchie** at AT&T Bell laboratories in 1972.

Advantages/Features of C Language

C language is very popular language because of the following features:

1. C is structured Programming Language
2. It is considered a high-level language because it allows the programmer to solve a problem without worrying about machine details.
3. It has wide variety of operators using which a program can be written easily to solve a given problem.
4. C is more efficient which increases the speed of execution and management of memory compared to low level languages.
5. C is machine independent. The program written on one machine will work on another machine.
6. C can be executed on many different hardware platforms

1.5 Structure of C Program

Comments/ Documentation section

Preprocessor Directives

Global declaration section

Void main() [Program Header]

{

Local Declaration part

Execution part

Statement 1

Statement n

}

User defined Functions

❖ Comments/Documentation Section

- Comments are short explaining the purpose of the program or a statement.
- They are non-executable statements which are ignored by the compiler.

The comment begins **with /* and ends with */**.

- The symbols /* and */ are called **comment line delimiters**.
- We can put any message we want in the comments.

Example: /* Program to compute Quadratic Equation */

Note: Comments are ignored by C compiler, i.e. everything within comment delimiters

❖ Preprocessor Directives

- Preprocessor Directives begins with a # symbol
- Provides instructions to the compiler to include some of the files before compilation starts.
- Some examples of header files are

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h> Etc...
```

❖ Global Declaration Section

There will be some variable which has to be used in anywhere in program and used in more than one function which will be declared as global.

❖ The Program Header

- Every program must contain a main() function.
- The execution always starts from main function.
- Every C program must have only one main function.

❖ Body of the program

- Series of statements that performs specific task will be enclosed within braces { and }
- It is present immediately after program header.

It consists of two parts

1. **Local Declaration part:** Describes variables of program

Ex: `int sum = 0;`

`int a , b;`

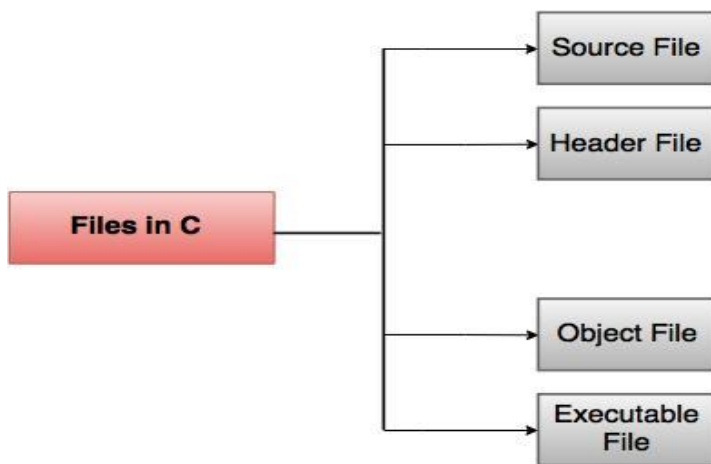
`float b;`

2. **Executable part:** Task done using statements.

Subroutine Section: All user defined functions that are called in main function should be defined.

1.6 Files used in a c program

“A File is a collection of data stored in the secondary memory”.



➤ Source Code File

1. This file includes the source code of the program.
2. The extension for these kind of files are '.c'. It defines the main and many more functions written in C.
3. main() is the starting point of the program. It may also contain other source code files.

➤ Header Files

They have an extension '.h'. They contain the C function declarations and macro definitions that are shared between various source files.

Advantages of header files:

1. At times the programmer may want to use the same subroutines for different programs. To do this, he would just compile the code of the subroutine once and link to the resulting object file in any file in which the functionalities of this subroutine are required.
2. At times the programmer may want to change or add the subroutines and reflect those changes in all the programs. For doing this, he will have to only change the source file for the subroutines, recompile the source code and then recompile and re-link the program.

This tells us that including a header file will make it easier at all levels of the program. If we need to modify anything then changes are made only in the subroutines after which all the changes will be reflected.

Standard header files

C provides us with some standard header files which are available easily.

Common standard header files are:

- i) **string.h** – used for handling string functions.
- ii) **stdlib.h** – used for some miscellaneous functions.
- iii) **stdio.h** – used for giving standardized input and output.
- iv) **math.h** – used for mathematical functions.
- v) **alloc.h** – used for dynamic memory allocation.
- vi) **conio.h** – used for clearing the screen.

The header files are added at the start of the source code so that they can be used by more than one function of the same file.

➤ **Object files**

1. They are the files that are generated by the compiler as the source code file is processed.
2. These files generally contain the binary code of the function definitions.
3. The object file is used by the linker for producing an executable file for combining the object files together. It has a '.o' extension.

➤ **Executable file**

1. This file is generated by the linker.
2. Various object files are linked by the linker for producing a binary file which will be executed directly.
3. They have an '.exe' extension.

1.7 Compiler

What is a compiler?

A compiler is a special program that translates a programming language's source_code into machine code, byte code or another programming language.

1.8 Compiling and executing c

Execution of C Program

Execution of a C program involves four steps

① Creating the program/Source code

Source code is a file with C programming instructions in high level language (C program written by the user). To create source code we use any text editor to write the program instructions. The instructions written in the source code must follow the C programming language rules. This will be saved with .c extension.

② Compiling the program

Compilation is the process of converting high level language instructions into low level language instructions. On receiving a source file, the compiler first checks for the errors. If there are any errors then compiler returns list of errors, if there are no errors then the source code is converted into object code and stores it as file with .obj extension.

③ Linking the program:

It is linking the program with functions that are needed from the C library. Object code generated by the compiler is given to the linker. The linker combines both the object code and specified header file code and generates an Executable file with .exe extension.

④ Executing the program

After completing compilation successfully an executable file is created with .exe extension. The processor can understand this .exe file content so that it can perform the task specified in the source file.

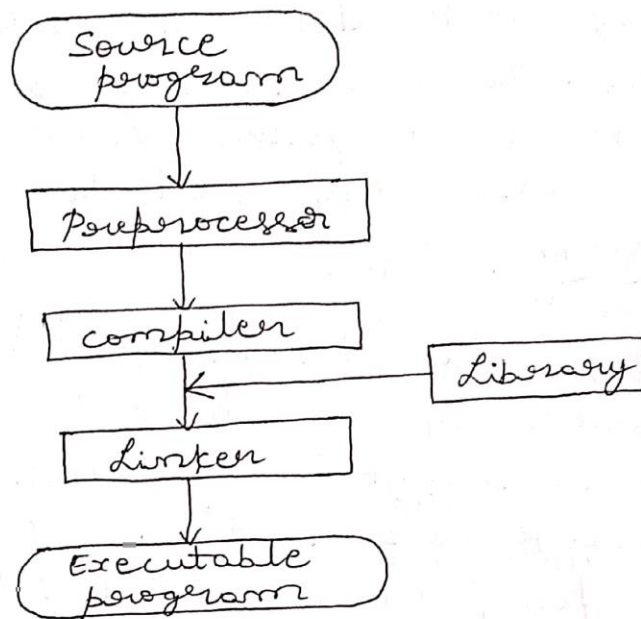


Fig: Compilation and linking of C program

Execution steps in Windows system

Creating Source Code:

The following steps are used to create source code file in Windows OS...

- click on File → New in C Editor Window
- Type the program
- Save it as Filename.c (use shortcut key F2 to save)

Compile source code and Linking (Alt + F9)

We use the shortcut key Alt + F9 to compile a C program in Turbo C. Whenever we press Alt + F9, the source file is going to

Compiling and Linking

Compilation can be done using cc command as
cc filename.c

If there are no errors then the file with extension .o is created, that is object code. Linking is automatically done. The compiled and linked program is called executable code and is stored automatically in another file named a.out

If there is a need of linking mathematical functions we should write as
cc filename.c -lm

Executing / Running Executable file

Execution can be done using command
./a.out

It will load executable code into computer memory and execute the instructions.

be submitted to the compiler. On receiving the source file, the compiler first checks for the errors. If there are any errors then compiler returns list of errors, if there are no errors then the source code is converted into object code and stores it as file with .obj extension. Then the object code is given to the linker. The linker combines both the object code and specified header file code and generates an Executable file with .exe extension.

Executing / Running Executable File (Ctrl+F9)

We use a shortcut key Ctrl + F9 to run a C program. Whenever we press Ctrl + F9, the .exe file is submitted to the CPU. On receiving .exe file, CPU performs the task according to the instruction written in the file. The result generated from the execution is placed in a window called User Screen.

1.9 VARIABLE

A variable is an identifier whose value can be changed during execution of the program.

In other words, a variable is a name given to a memory-location where the data can be stored.

Using the variable name, the data can be

→ stored in a memory-location and

→ accessed or manipulated

Rules for defining a variable

- 1) The first character in the variable should be a letter or an underscore
- 2) The first character can be followed by letters or digits or underscore
- 3) No extra symbols are allowed (other than letters, digits and underscore)
- 4) Length of a variable can be up to a maximum of 31 characters
- 5) Keywords should not be used as variable-names

Valid variables: a, principle_amount, sum_of_digits

Invalid variables: 3fact //violates rule1

sum=sum-of-digits62\$ //violates rule3

for int if //violates rule5

Declaration of Variable

- The declaration tells the compiler

→ what is the name of the variable used

→ what type of data is held by the variable

- The syntax is shown below:

data_type v1,v2,v3;

where v1,v2,v3 are variable-names data_type can be int, float or char

- For ex:

int a, b, c; float x, y, z;

Initialization of Variable

- The variables are not initialized when they are declared. Hence, variables normally contain garbage values and hence they have to be initialized with valid data.

- Syntax is shown below: data_type var_name=data;

where data_type can be int, float or char var_name is a name of the variable ,

= is assignment operator

data is the value to be stored in variable

- For ex: int a=10; float pi=3.1416; char c='z';

1.10 CONSTANTS

- A constant is an identifier whose value remains fixed throughout the execution of the program.

- The constants cannot be modified in the program.

- For example: 1, 3.14512, „z“, “sce”

Different types of constants are:

1) Integer Constant

An integer is a whole number without any fraction part.

There are 3 types of integer constants:

i) Decimal constants (0 1 2 3 4 5 6 7 8 9)

For ex: 0, -9, 22

ii) Octal constants (0 1 2 3 4 5 6 7)

For ex: 021, 077, 033

iii) Hexadecimal constants (0 1 2 3 4 5 6 7 8 9 A B C D E F) For ex: 0x7f, 0x2a, 0x521

2) Floating Point Constant

- The floating point constant is a real number.
- The floating point constants can be represented using 2 forms:

i) Fractional Form

A floating point number represented using fractional form has an integer part followed by a dot and a fractional part.

For ex: 0.5, -0.99

ii) Scientific Notation (Exponent Form)

The floating point number represented using scientific notation has three parts namely: mantissa, E and exponent.

For ex: 9.86E3 imply 9.86×10^3

3) Character Constant

- A symbol enclosed within a pair of single quotes (') is called a character constant.
- Each character is associated with a unique value called an ASCII (American Standard Code for Information Interchange code).
- For ex: '9', 'a', '\n'

4) String Constant

- A sequence of characters enclosed within a pair of double quotes (") is called a string constant.
- The string always ends with NULL (denoted by \0) character.
- For ex: "9" "a" "sri" "\n"

5) Escape Sequence Characters

- An escape sequence character begins with a backslash and is followed by one character.
- A backslash (\) along with some characters give rise to special print effects by changing (escaping) the meaning of some characters.
- The complete set of escape sequences are:

Escape Sequences Character

\b	Backspace
\f	Form feed
\n	Newline

\r	Return
\t	Horizontal tab
\v	Vertical tab
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark
\?	Question mark
\0	Null character

1.11 INPUT/OUTPUT STATEMENTS IN C

There are many library functions for input and output in C language.

For ex: `getchar( )`, `putchar( )`, `scanf( )`, `printf( )`

For using these functions in a C-program there should be preprocessor statement `#include<stdio.h>`.

UNFORMATTED I/O

getchar() and putchar()

- `getchar()` is used to
 - read a character from the keyboard and
 - store this character into a memory-location
- You have to press ENTER key after typing a character.

The syntax is shown below:

```
char variable_name = getchar( );
```

For ex: `char z;` `z= getchar( );`

`putchar()` is used to display a character stored in the memory-location on the screen.

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
char x;
```

```
char y="n";
```

```
printf("enter one letter terminated by ENTER key \n");
```

```
x = getchar();
```

```
putchar(y); // same as printf("%c",z);
```

```
}
```

Output:

enter one letter terminated by ENTER key

m

n

gets() and puts()

- `gets()` is used to
 - read a string from the key board and

→ store the string in memory-locations

- puts() is used to display a string stored in memory-locations on the screen.

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
char string[20];
```

```
printf("enter a string \n");
```

```
gets(string);
```

```
puts(string); // same as printf("%s",string);
```

```
}
```

Output:

enter a string

Sapthagiri

Sapthagiri

Disadvantage of Unformatted I/O

- It is not possible to read/print any other data except characters i.e. it is not possible to read/print integer numbers, floating point numbers etc.

FORMATTED I/O

scanf()

- The scanf function does following tasks:

→ Scan a series of input fields one character at a time

→ Format each field according to a corresponding format-specifier passed in format-string (format means convert).

→ Store the formatted input at an address passed as an argument i.e. address-list

Syntax is shown below:

```
n=scanf("format-string", address-list);
```

where format-string contains one or more format-specifiers

address-list is a list of variables. Each variable name must be preceded by &

For ex: n=scanf("%d %f %c", &x, &y, &z);

Format specifiers Meaning

%d an int argument in decimal

%ld a long int argument in decimal

%c a character

%s a string

%f a float or double argument

%e same as %f, but use exponential notation

`%o` an int argument in octal (base8)

`%x` an int argument in hexadecimal (base16)

printf

The printf function does following tasks:

- Accept a series of arguments
- Apply to each argument a format-specifier contained in the format-string
- Output the formatted data to the screen

The syntax is shown below:

```
n=printf("format-string", variable-list);
```

where format-string contains one or more format-specifiers variable-list contains names of variables

For ex: `n=printf("%d %f %c", x, y, z );`

- Example: Program to read your age and display the same on the screen

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
int age;
```

```
printf("enter your age: \n");
```

```
scanf("%d", age);
```

```
printf("your age is = %d years ", age);
```

```
}
```

Output:

enter your age: 21

your age is = 21 years